

3SE2 – Übung 6

Themen: Benutzerverwaltung, Login, Registrierung, Pull-Requests

Die Übung darf alternativ im Einzelprojekt umgesetzt werden.

Folgendes wird erwartet.

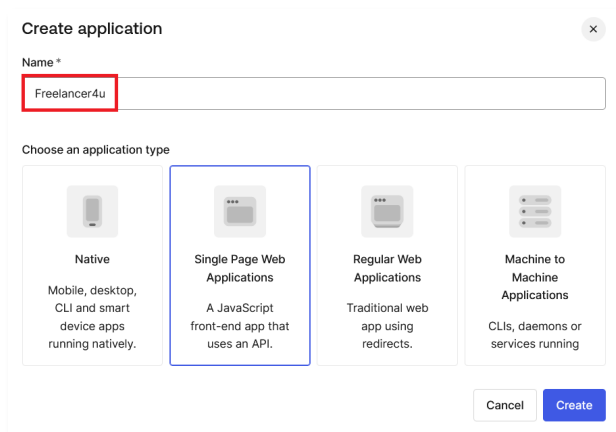
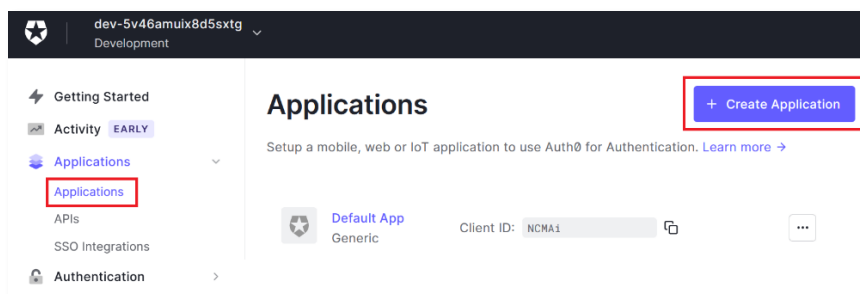
- Das Projekt enthält einen Login mit Signup welcher Auth0 verwendet
- Teile des Frontends werden nur mit Login angezeigt. Die entsprechenden Seiten laden Daten aus dem Backend.
- Das Backend ist mit Login abgesichert.
- Gewisse Teile des Frontends und zugehörige Endpoints im Backend können nur von einer bestimmten Rolle verwendet werden.
- Das Projekt enthält einen Branch, der mit Pull-Request gemerged wurde.

In dieser Übung machen wir weiter mit dem Freelancer-Jobportal und erweitern dieses mit einem Login und einer rollenbasierten Benutzerverwaltung. Im ersten Teil implementieren die Funktion, um eingeloggte und nicht eingeloggte User zu unterscheiden. Im zweiten Teil werden wir Rollen mit unterschiedlichen Berechtigungen umsetzen

1. Login implementieren

a) Erstelle auf <https://auth0.com/> einen Account.

b) Erstelle eine neue Single Page Applikation



c) Kopiere die Domain und die Client-ID und speichere diese in einer Textdatei.

The screenshot shows the 'Freelancer4u' settings page. The 'Settings' tab is selected. Under 'Basic Information', the following fields are visible:

- Name ***: Freelancer4u
- Domain**: dev-5v46 us.auth0.com (The domain 'us.auth0.com' is highlighted with a red box and a copy icon.)
- Client ID**: PWbQ (The Client ID 'PWbQ' is highlighted with a red box and a copy icon.)
- Client Secret**: A masked field with a copy icon. A note below states: 'The Client Secret is not base64 encoded.'

d) Konfiguriere die «Application URIs» sowie «Cross-Origin Authentication»:

Application URIs

Application Login URI

<https://myapp.org/login>

In some scenarios, Auth0 will need to redirect to your application's login page. This URI needs to point to a route in your application that should redirect to your tenant's `/authorize` endpoint. [Learn more](#)

Allowed Callback URLs

<http://localhost:5173/>

Comma-separated list of allowed callback URLs for redirecting users after login. Unlisted URLs will be rejected. [Learn more about application URIs](#)
You can use [Organization URL](#) parameters in these URLs.

Allowed Logout URLs

<http://localhost:5173/>

Comma-separated list of allowed logout URLs for redirecting users post-logout. You can use wildcards at the subdomain level (`*.google.com`). Query strings and hash information are not taken into account when validating these URLs. [Learn more about logout](#)

Allowed Web Origins

<http://localhost:5173/>

Comma-separated list of allowed origins for use with [Cross-Origin Authentication](#), [Device Flow](#), and [web message response mode](#), in the form of `<scheme>://<host>[:<port>]`, such as `https://login.mydomain.com` or `http://localhost:3000`. You can use wildcards at the subdomain level (e.g.: `https://*.contoso.com`). Query strings and hash information are not taken into account when validating these URLs.

Cross-Origin Authentication

Allow Cross-Origin Authentication



When allowed, [cross-origin authentication](#) allows applications to make authentication requests when the Lock widget or custom HTML is used.

Allowed Origins (CORS)

http://localhost:5173/

List additional origins allowed to make [cross-origin resource sharing \(CORS\)](#) requests. Allowed callback URLs are already included in this list.

- URLs can be comma-separated or added line-by-line
- Use wildcards (*) at the subdomain level (e.g. https://*.contoso.com)
- Query strings and hash information are ignored
- [Organization URL](#) placeholders are supported

Cross-Origin Verification Fallback URL

https://domain.tld/path

Fallback URL when third-party cookies are not enabled in the browser. URL must use https and be in the same domain as the embedded login widget.

Anmerkung: <http://localhost:5173/> ist die Adresse des Frontends in unserer Entwicklungsumgebung.

- e) Passe unter «Advanced Settings» die «Grant Types» an und speichere am Schluss alle Änderungen.

Anmerkung: Dies ist nötig, damit wir uns später im Frontend direkt via Username und Passwort einloggen können (ohne das Login-Popup von Auth0).

Advanced Settings

Application Metadata Device Settings OAuth **Grant Types** WS-Federation Certificates Endpoints

⚠ Using **Password** or **MFA** grant types with public applications is not recommended.
To use the **Client Credentials** grant you have to set a **Token Endpoint Auth Method** other than "**none**".
See our [documentation](#) for more information.

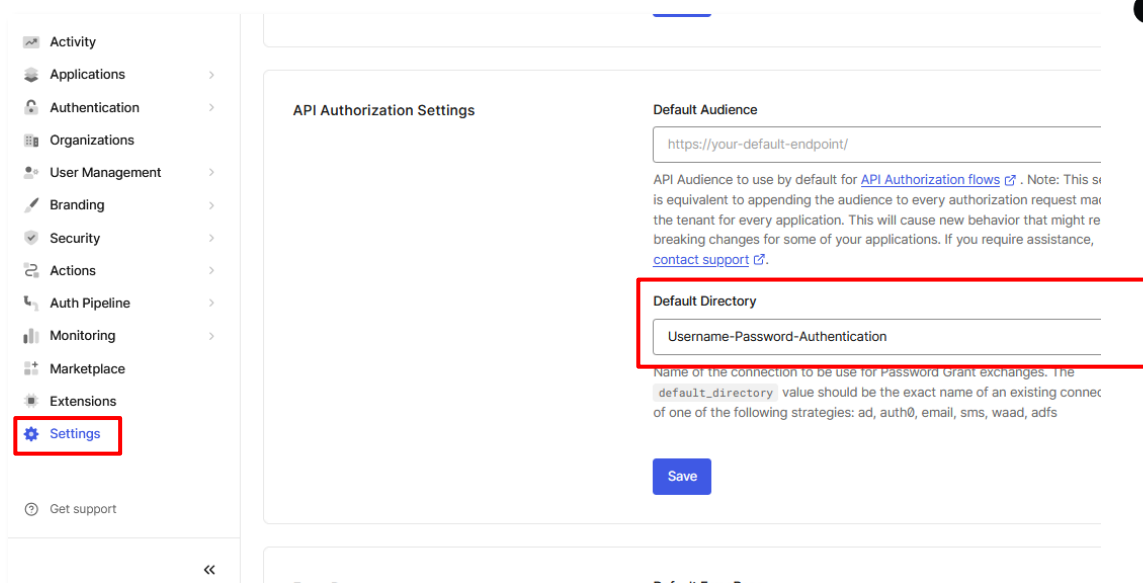
Grant Types

☒ Implicit ☒ Authorization Code ☒ Refresh Token ☐ Client Credentials ☒ Password

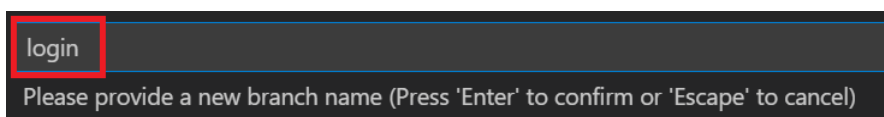
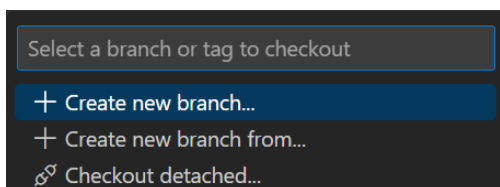
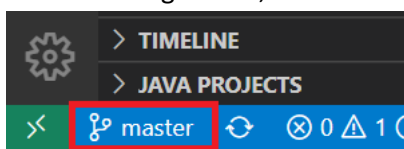
☐ MFA ☐ Passwordless OTP

Save changes CTRL + Enter Cancel **Save Changes**

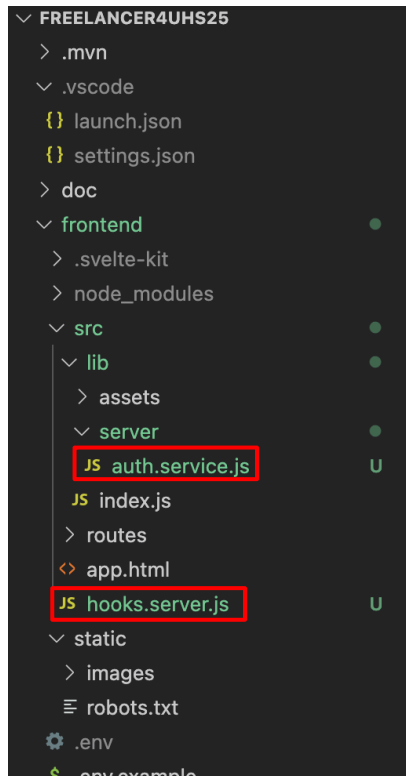
Setze bei den allgemeinen Settings unter «API Authorization Settings» den Wert für «Default Directory» auf «Username-Password-Authentication» (ohne Anführungszeichen). Drücke auch hier den **Save**-Button.



- f) Nun beginnen wir mit den Anpassungen unserer Freelancer-Applikation. Die Änderungen sind etwas umfangreicher, deshalb machen wir einen Branch.



- g) Kopiere die Datei **hooks.server.js** in den Ordner **frontend/src** und **auth.service.js** in den Ordner **frontend/src/lib/server** (der Ordner **server** existiert noch nicht und muss erstellt werden). Beide Dateien können von Moodle heruntergeladen werden.



Erklärungen zu den eingefügten Dateien:

- **auth.service.js** benötigt keine Anpassungen. Diese Datei enthält die Funktionen zum Registrieren, Einloggen und Ausloggen sowie eine Funktion, um Informationen zum eingeloggten User von Auth0 abzufragen.
Das beim Einloggen von Auth0 erhaltene JWT wird in einem Cookie gespeichert. Auf diese Weise sind alle benötigten Informationen zum eingeloggten User bei jedem Request verfügbar.
- **hooks.server.js** muss ebenfalls nicht geändert werden. Dieser Code wird bei jedem Request ausgeführt und sorgt dafür, dass die Userinformationen (JWT, Username, etc.) auch im serverseitigen Teil der SvelteKit-App verfügbar sind. Somit kann beispielsweise in einer load-Funktion auf das JWT zugegriffen werden, das dann ans Java-Backend weitergeleitet wird.

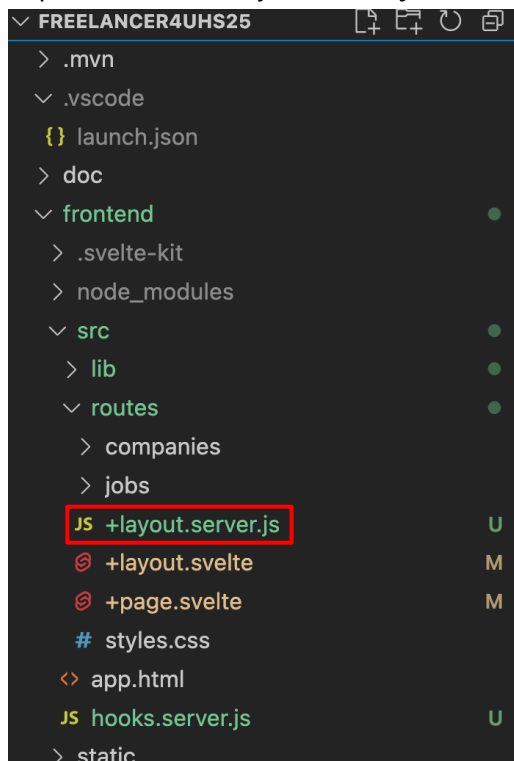
Anmerkung: Diese Konzepte (Speichern von Daten in Cookies, Weiterleitung von JWTs, usw.) sowie die entsprechende die Architektur der SvelteKit-App werden in der Vorlesung in Woche 9 detaillierter behandelt. Du musst den Code noch nicht vollständig verstehen.

- h) Ergänze die Datei **frontend/.env** wie unten gezeigt und füge die in Schritt c) kopierte Domain und Client-ID von Auth0 ein (ohne Anführungszeichen!)

```
# Backend API URL
API_BASE_URL=http://localhost:8080

# Auth0 Configuration
AUTH0_DOMAIN=...
AUTH0_CLIENT_ID=...
```

- i) Kopiere die Datei **+layout.server.js** von Moodle in den Ordner **frontend/src/routes**



j) Mache die folgenden Änderung an der Navbar in **routes/+layout.svelte**.

```

<script>
  import favicon from "$lib/assets/favicon.svg";
  import "./styles.css";
  let { data, children } = $props();
  let { user, isAuthenticated } = data;
</script>

<svelte:head>
  <link rel="icon" href={favicon} />
</svelte:head>

<nav class="navbar navbar-expand-lg bg-light">
  <div class="container-fluid">
    <a class="navbar-brand" href="/">Freelancer4U</a>
    <button
      class="navbar-toggler"
      type="button"
      data-bs-toggle="collapse"
      data-bs-target="#navbarNav"
      aria-controls="navbarNav"
      aria-expanded="false"
      aria-label="Toggle navigation"
    >
      <span class="navbar-toggler-icon"></span>
    </button>
    <div class="collapse navbar-collapse" id="navbarNav">
      <ul class="navbar-nav me-auto mb-2 mb-lg-0">
        {#if isAuthenticated}
          <li class="nav-item">
            <a class="nav-link" aria-current="page" href="/companies">
              Companies</a>
          </li>
        {/if}
        {#if isAuthenticated}
          <li class="nav-item">
            <a class="nav-link" href="/jobs">Jobs</a>
          </li>
        {/if}
      </ul>
      <div class="d-flex">
        {#if isAuthenticated}
          <span class="navbar-text me-2">{user.name}</span>
          <form method="POST" action="/logout" style="display: inline;">
            <button type="submit" class="btn btn-primary">Log Out</button>
          </form>
        {:else}
          <a href="/login" class="btn btn-primary me-2">Login</a>
          <a href="/signup" class="btn btn-outline-primary">Sign Up</a>
        {/if}
      </div>
    </div>
  </div>
</nav>

<div class="container mt-3">
  {@render children()}
</div>

```

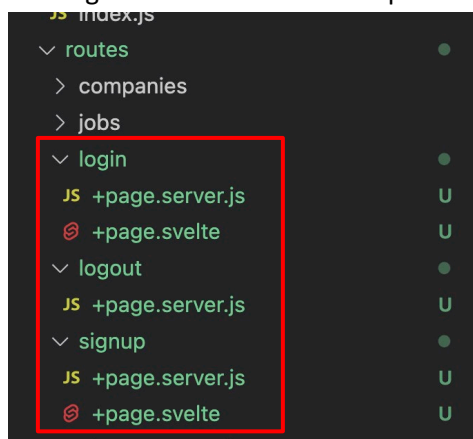
Daten aus der load-Funktion (+layout.server.js)

Klassen angepasst, damit der Logout Button rechts ausgerichtet ist

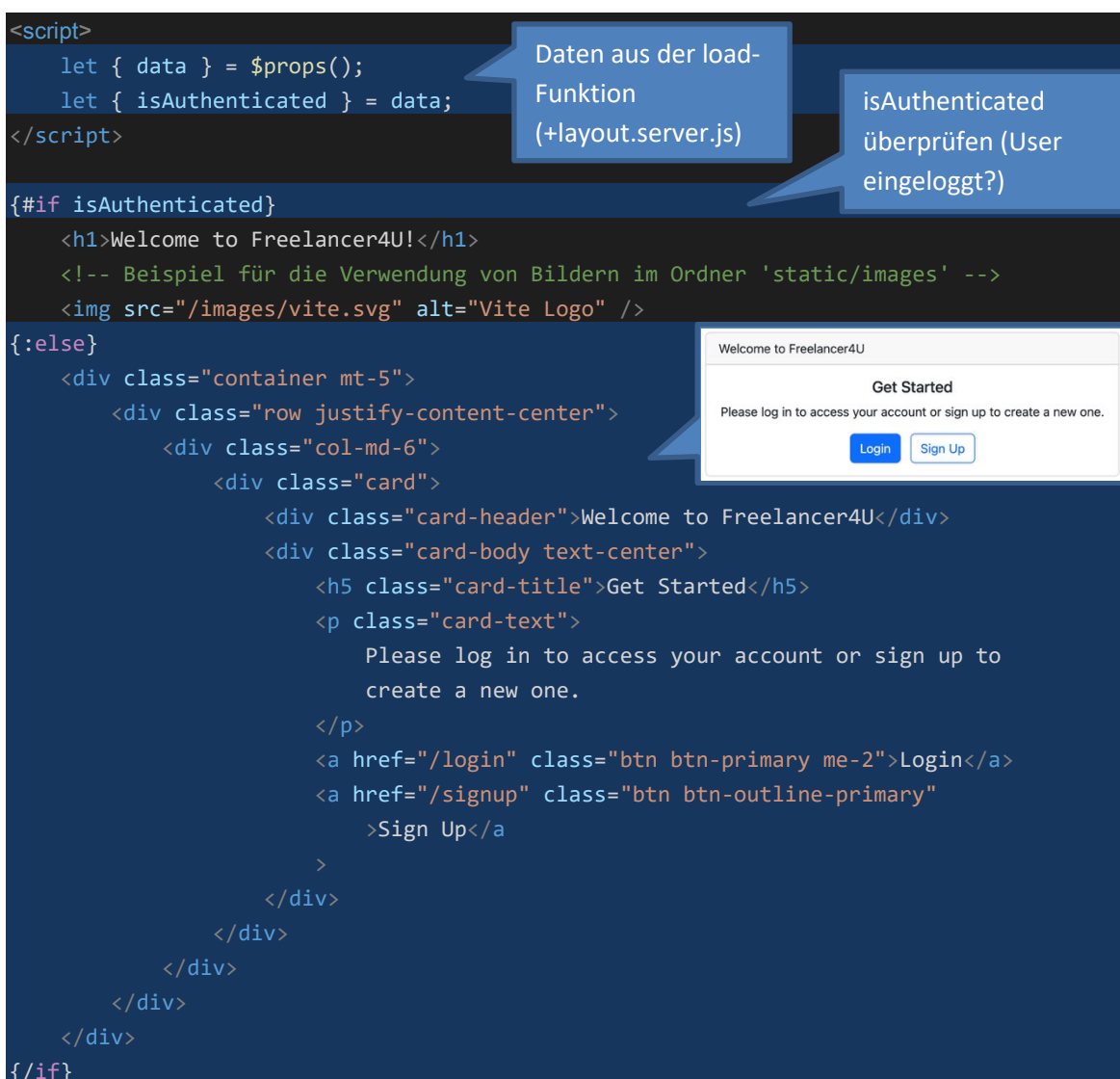
isAuthenticated überprüfen (User eingeloggt?)

Logout/Login-Buttons und anzeigen des Usernamens

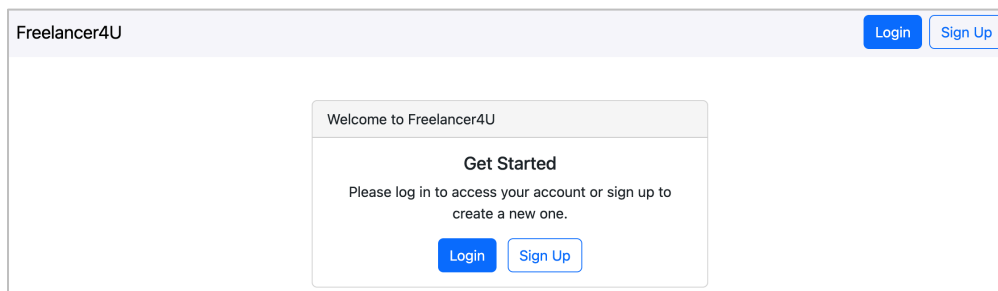
- k) Nun fehlen nur noch die Formulare für Registrierung und An-/Abmeldung. Kopiere die 3 benötigten Ordner mit den entsprechenden Dateien von Moodle in dein Projekt:



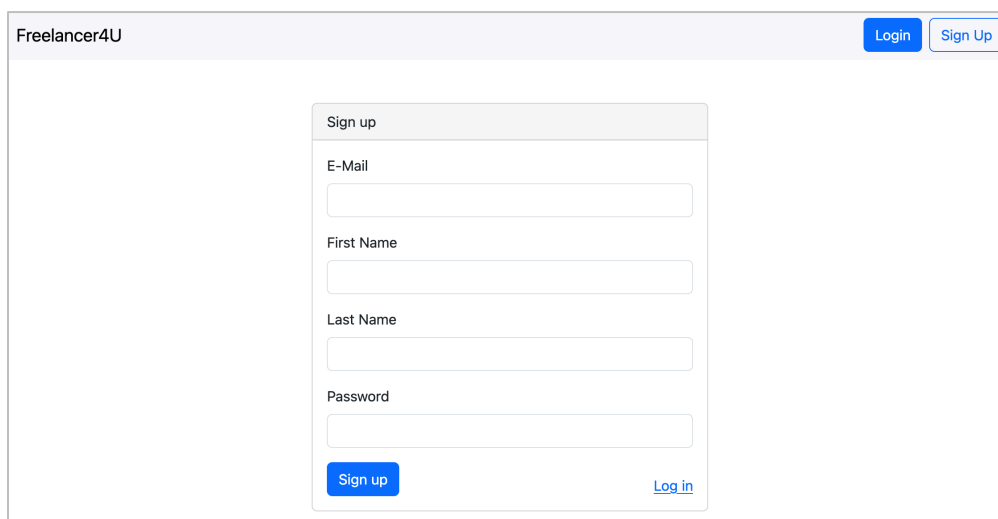
Passe ausserdem die Hauptseite *frontend/src/routes/+page.svelte* wie folgt an:



- l) Starte das Java-Backend und das SvelteKit-Frontend (mit `npm run dev`). Teste die Änderungen. Wird die Applikation aufgerufen, sollten die Tabs «Companies» und «Jobs» nicht mehr angezeigt werden. Stattdessen solltest du folgende Page sehen.



Klicke auf «Sign up» und erstelle einen Account. Du kannst eine beliebige E-Mail-Adresse verwenden, beachte aber, dass Auth0 nach erfolgter Registrierung eine Nachricht an diese Adresse schickt. Verwende **nicht** das E-Mail deines Auth0-Accounts. Merke dir das Passwort 😊.



Falls das Sign-up **nicht erfolgreich** war, kann dies verschiedene Gründe haben. Prüfe zunächst, ob der Request korrekt geschickt werden konnte. Fehlermeldungen werden im **Terminal von VS Code** angezeigt.

Die genaue Fehlerursache ist in den Logs von Auth0 ersichtlich. Öffne die Logs wie unten gezeigt und schau dir die Description beim Fehler an. Typische Fehler sind «Password is too weak» und «The user already exists».

Freelancer4U

CompaniesJobs

david@zhaw.ch

Log Out

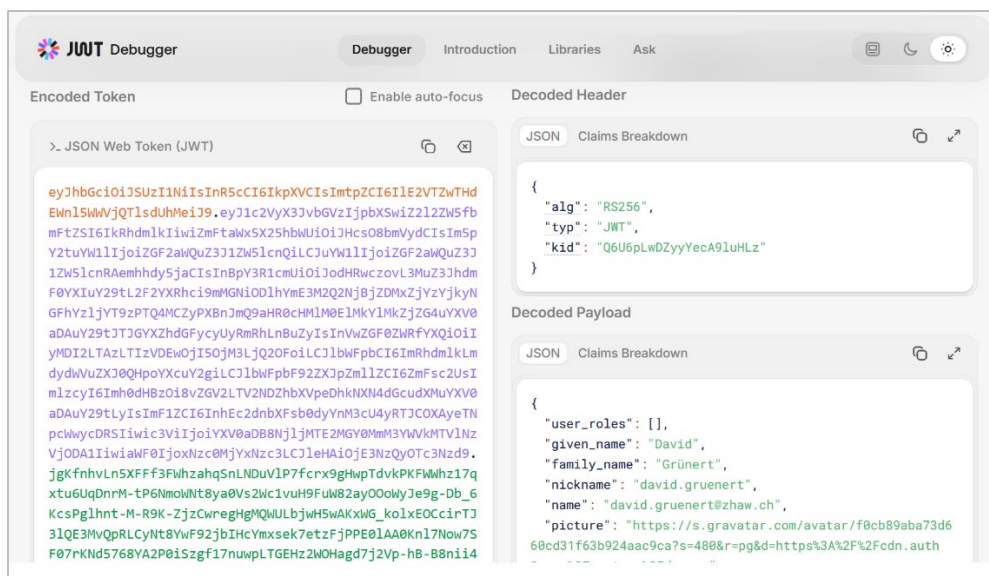
Welcome to Freelancer4U!



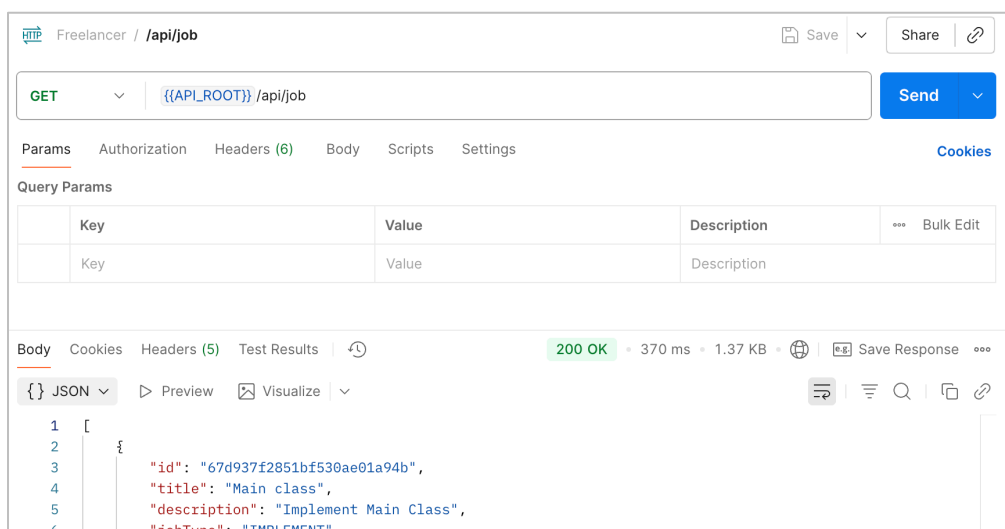
- m) Committe deine Änderungen (Commit-Message: «added login to frontend») auf GitHub.
Bestätige das Erstellen des Remote-Branchs, falls du danach gefragt wirst.
- n) Als nächstes wollen wir uns das JWT genauer anschauen. In **auth.service.js** wird dieses zu Testzwecken auf die Konsole geschrieben. Kopiere dein JWT aus dem Terminal **von VS Code**. Falls das Token noch nicht erscheint, musst du dich aus- und wieder einloggen.

[illegible]

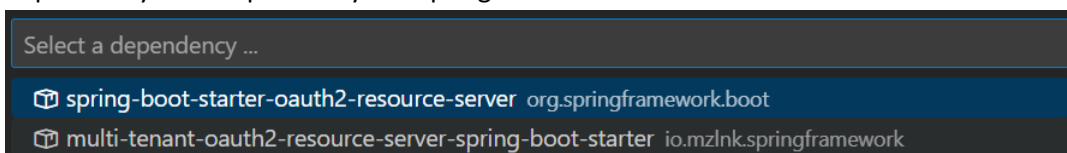
Rufe die Seite <https://jwt.io/> auf und ersetze das vorhandene Token im Bereich «Encoded» mit deinem kopierten JWT. Prüfe was passiert, wenn du ein Zeichen im JWT änderst.



Wir haben nun das Frontend so geändert, dass z.B. der Tab «Jobs» nur für eingeloggte User sichtbar ist. Das allein bietet jedoch keinen Schutz davor, dass Unberechtigte auf die Jobs zugreifen können. Das Backend erlaubt nach wie vor unautorisierten Zugriff, was sich mit Postman leicht prüfen lässt. Teste es bei deiner Applikation!



- o) Beginnen wir also mit den Anpassungen des Backends. Füge zuerst mit «Maven: Add dependency» die dependency zu «spring-boot-starter-oauth2-resource-server» hinzu.



Entferne anschliessend die Versionsnummer aus dem POM-File:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
</dependency>
```

Vergiss nicht anschliessend das POM zu speichern und den Classpath zu synchronisieren.

p) Ergänze **.vscode/launch.json** mit der Auth0-Domain aus Schritt c):

```
{
  "configurations": [
    {
      "type": "java",
      "name": "Spring Boot-Freelancer4uApplication<freelancer4u>",
      "request": "launch",
      "cwd": "${workspaceFolder}",
      "mainClass": "ch.zhaw.freelancer4u.Freelancer4uApplication",
      "projectName": "freelancer4u",
      "args": "",
      "envFile": "${workspaceFolder}/.env",
      "env": {
        "MONGODB_URI": "...",
        "AUTH0_DOMAIN": "..."
      }
    }
  ]
}
```

(Komma am Ende der vorhergehenden Zeile nicht vergessen.)

und ergänze folgende Zeile in **application.properties**:

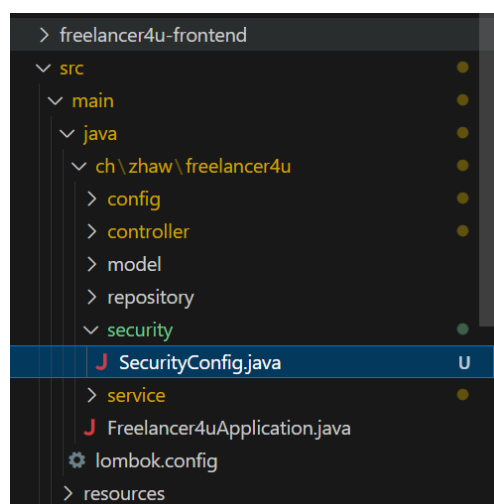
```
spring.application.name=freelancer4u

spring.data.mongodb.uri=${MONGODB_URI}
spring.data.mongodb.database=FreelancerHS25

spring.security.oauth2.resourceserver.jwt.issuer-uri=https://${AUTH0_DOMAIN}/
```

WICHTIG: Weil *launch.json* nur beim Start gelesen wird, musst du jetzt **VS Code neu starten**.

q) Erstelle den Unterordner «security» und kopiere die Datei **SecurityConfig.java** hinein (Dateien als Download auf Moodle).

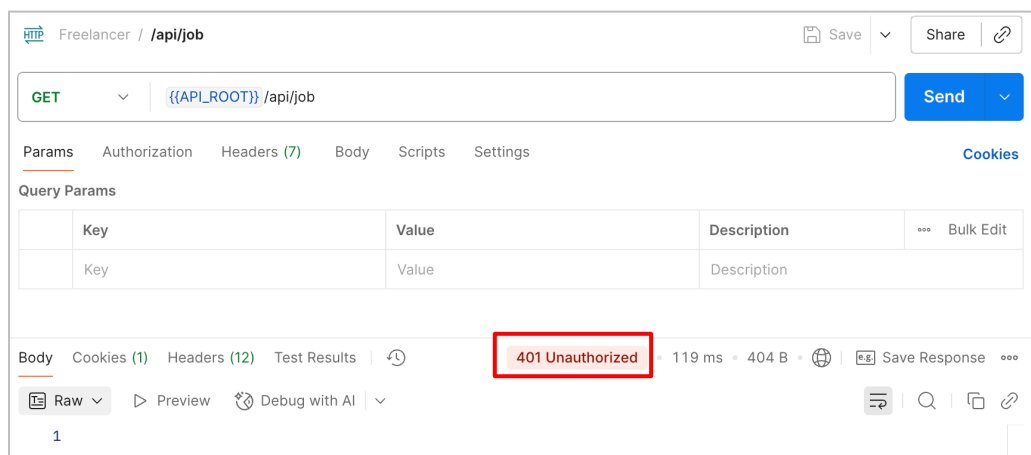


Erklärungen zur Datei **SecurityConfig**: Mit der Annotation `@EnableWebSecurity` wird spring-security aktiviert. In der `filterChain` wird festgelegt, welche Endpoints mit Authentisierung geschützt werden. Im Beispiel unten wird für alle Endpoints, die mit `/api` beginnen, Authentisierung verlangt.

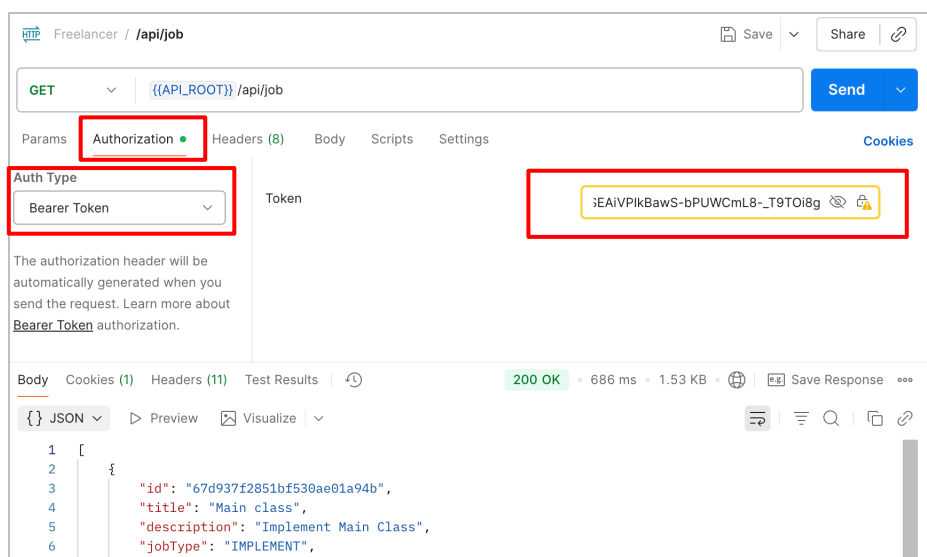
```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    http
        .authorizeHttpRequests(authorize -> authorize
            .requestMatchers("/").permitAll()
            .requestMatchers("/api/**").authenticated()
            .requestMatchers("/**").permitAll()
        )
        .oauth2ResourceServer((oauth2) -> oauth2.jwt(Customizer.withDefaults()));

    return http.build();
}
```

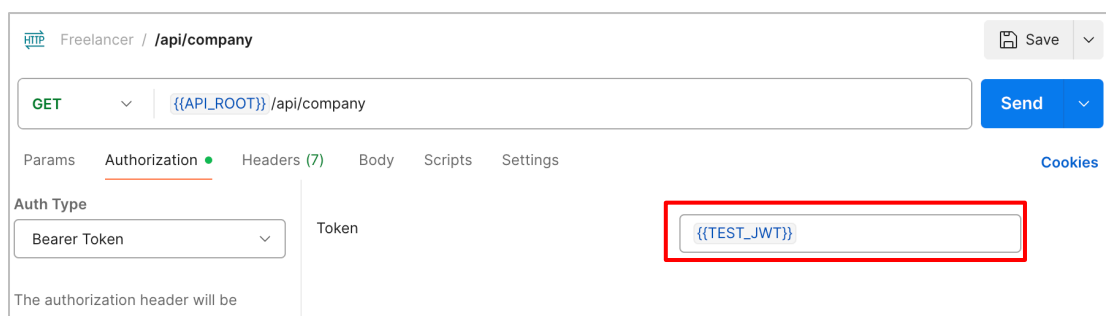
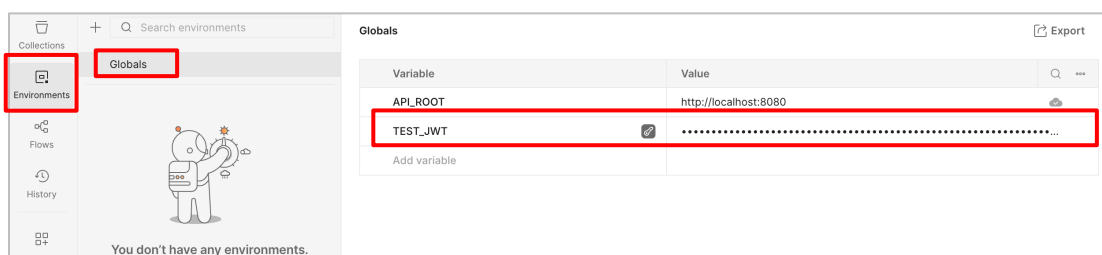
- r) Nun haben wir die api-Endpoints abgesichert. Mache einen Restart des Springboot-Projekts und prüfe mit Postman, ob uns der direkte Zugriff nun verwehrt bleibt.



- s) Ein Zugriff ist nun nur noch mit dem JWT möglich. Kopiere nochmal das JWT aus der Konsole des Browsers und füge dies wie unten gezeigt in Postman ein. Nun sollte der Request wieder funktionieren.

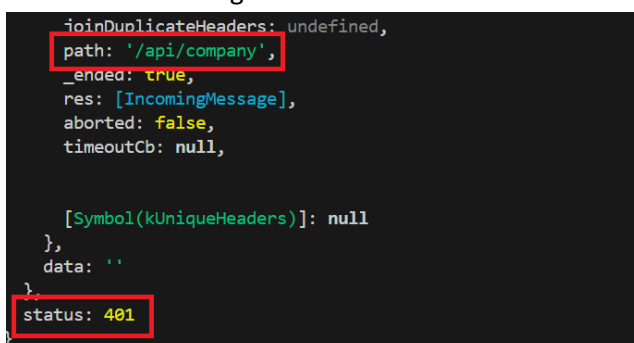


Das JWT hat nur eine beschränkte Gültigkeitsdauer. Damit wir das JWT nicht manuell bei allen Request aktualisieren müssen, speichern wir dieses in einer globalen Variable. Vergiss den Save-Button nicht.



Prüfe, ob der Request immer noch funktioniert.

- t) Weil die Ressourcen jetzt abgesichert sind, können im Frontend weder Jobs noch Companies angezeigt werden. Wenn du z.B. die Seite mit den Companies neu aufrufst, gibt es in der Konsole eine Fehlermeldung und die Tabelle bleibt leer.



- u) Damit das Frontend wieder funktioniert, muss das JWT bei jedem Request auf geschützte Ressourcen mitgeschickt werden. Das folgende Beispiel zeigt die erforderlichen Anpassungen für einen Request aus *routes/companies/+page.server.js*.

```
...
export async function load({ locals }) {
  const jwt_token = locals.jwt_token;

  if (!jwt_token) {
    return {
      companies: []
    };
  }

  try {
    const response = await axios({
      method: "get",
      url: `${API_BASE_URL}/api/company`,
      headers: { Authorization: "Bearer " + jwt_token },
    });
    return {
      companies: response.data
    };
  } catch (axiosError) {
    console.log('Error loading companies:', axiosError);
  }
}

export const actions = {
  createCompany: async ({ request, locals }) => {
    const jwt_token = locals.jwt_token;

    if (!jwt_token) {
      throw error(401, 'Authentication required');
    }

    const data = await request.formData();
    const company = {
      name: data.get('name'),
      email: data.get('email')
    };

    try {
      await axios({
        method: "post",
        url: `${API_BASE_URL}/api/company`,
        headers: {
          "Content-Type": "application/json",
          Authorization: "Bearer " + jwt_token,
        },
        data: company,
      });
      return { success: true };
    } catch (error) {
      console.log('Error creating company:', error);
      return { success: false, error: 'Could not create company' };
    }
  }
};
```

In der load-Funktion muss auf die in *hooks.server.js* gesetzten *locals* zugegriffen werden.

JWT auslesen

JWT im Request mitschicken.

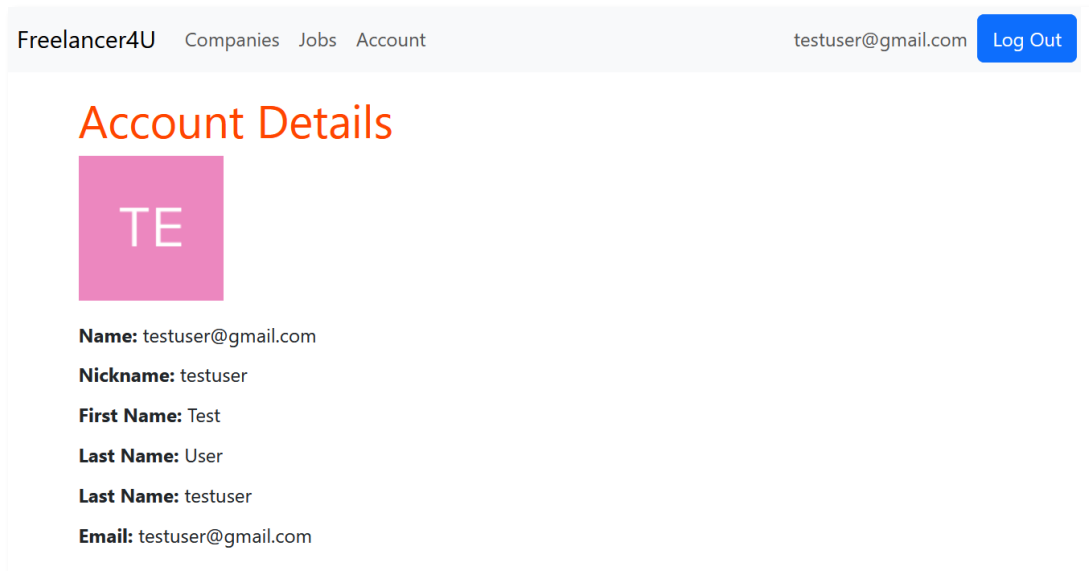
In der Action muss auf die in *hooks.server.js* gesetzten *locals* zugegriffen werden.

JWT auslesen

JWT im Request mitschicken.

Teste, ob die Seite «Company» im Frontend wieder wie gewünscht funktioniert. Eventuell muss die Seite neu geladen werden (Ctrl-Shift-R unter Windows). Möglicherweise musst du dich auch aus- und wieder einloggen.

- v) Die Requests für Companies funktionieren jetzt. Passe **alle restlichen Requests für die Jobs** (GET und POST) auf /api/... im Frontend so an, dass das JWT mitgeschickt wird. Prüfe, ob das Frontend wieder wie gewünscht funktioniert.
- w) Ergänze zum Abschluss die Applikation mit einer Page, welche die Details des Accounts anzeigt.



Folgende Schritte sind dafür notwendig:

- Füge eine neue Page unter **routes/account/+page.svelte** hinzu mit folgendem Inhalt:

```
<script>
  let { data } = $props();
  let { user, isAuthenticated } = data;
</script>

<h1>Account Details</h1>

{#if isAuthenticated}
  <p><img src={user.picture} alt="" /></p>
  <p><b>Name:</b> {user.name}</p>
  <p><b>Nickname:</b> {user.nickname}</p>
  <p><b>First Name:</b> {user.given_name}</p>
  <p><b>Last Name:</b> {user.family_name}</p>
  <p><b>Email:</b> {user.email}</p>
{:else}
  <p>Not logged in</p>
{/if}
```

- Ergänze **routes/+layout.svelte** mit einem nav-item Eintrag für die neue Page. Dieser soll nur angezeigt werden, wenn der User eingeloggt ist.

- x) Committe deine Änderungen auf GitHub (Commit-Message: «added login to backend»).

2. Rollenbasierter Zugriff

Bisher unterscheidet die App nur zwischen eingeloggten und nicht eingeloggten BenutzerInnen. Gewisse Aktionen sollen jedoch nur von Mitarbeitern (Admins) ausgeführt werden können. Wir machen folgende Vereinfachung: Alle User ohne Rolle betrachten wir als *Freelancer* und allen Mitarbeitern geben wir die Rolle *Admin*.

- a) Wir verwenden Auth0, um die Rollen der BenutzerInnen zu verwalten. Erstelle als erstes eine Rolle für den Admin.

Roles

Create and manage Roles for your applications. Roles contain collections of Permissions and can be assigned to Users.

You don't have any roles yet.

Create roles to represent the types of users that access your applications. Assign permissions to those roles to control what users are allowed to do in your apps.

[+ Create Role](#)

New Role

Name *

admin

Description *

Administrator

[Cancel](#) [Create](#)

admin

Role ID ro1_NoW900upsTuGxC0m

[Settings](#) [Permissions](#) [Users](#)

Name *

admin

Description *

Administrator

[Save](#)

- b) Erstelle einen neuen User, um die Admin Rolle zu testen.

Users

[+ Create User](#)

Search for users

[Reset](#)

Name	Connection	Latest Login
david.gruenert@zhaw.ch david.gruenert@zhaw.ch	Username-Password-Authenti...	3 hours ago

Ordne anschliessend dem neu erstellten User die Admin-Rolle zu

Users

An easy to use UI to help administrators manage user identities including password resets, creating and provisioning, blo

Search for users Search by User

Name	Connection	Logins	Latest Login
GR gaud@zhaw.ch gru@zhaw.ch	Username-Password-Authenti...	3	

View Details

- Assign Roles
- Assign Permissions
- Send Verification Email
- Block
- Delete

Neu erstellter User

c) Damit die Rolle mit dem JWT mitgegeben wird, muss eine entsprechende Action erstellt werden.

Library

Custom Installed Modules

Custom

Actions allow you to leverage custom code within your trigger flows.

Create Action

Create Custom Action

Choose a template

Create Action

Name *

Add role to token

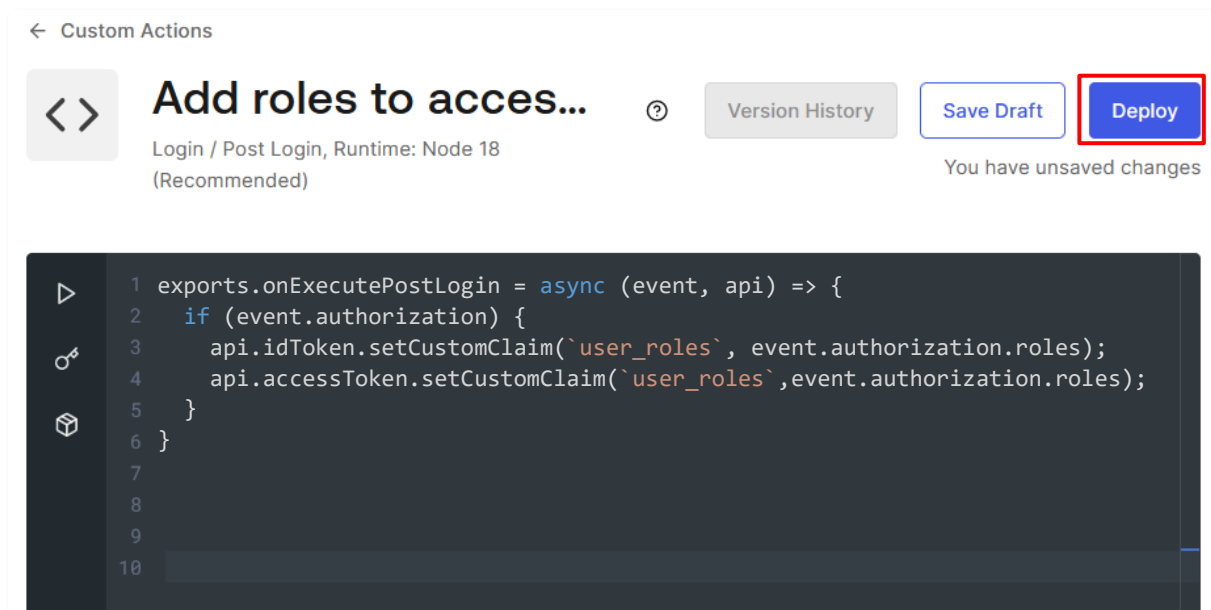
Trigger *

Login / Post Login

Runtime *

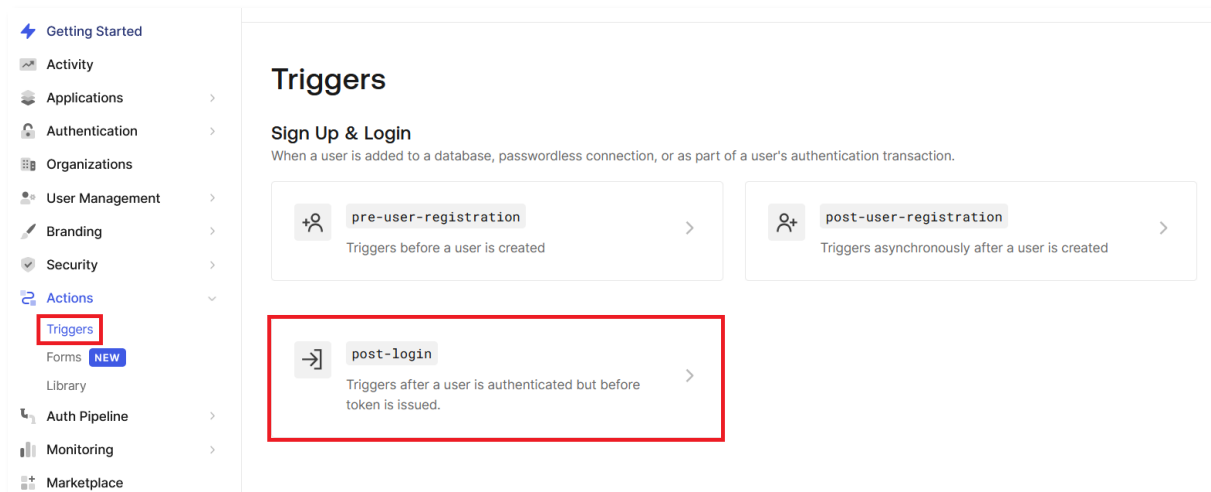
Node 22 (Recommended)

Cancel Create



Passe die Funktion wie oben gezeigt an. Ja, das kann man kopieren 😊. Klicke am Schluss auf «Deploy», um die Action zu aktivieren.

Damit die Action automatisch beim Einloggen ausgeführt wird, muss du sie einem post-login Trigger hinzufügen.



Füge die Action via Drag and Drop ein:

← Choose trigger

Post Login

Customize the login trigger for your applications.

ⓘ Apply

All changes are live

Start
User Logged In

Add role to access token

Complete
Token Issued

Add Action

Installed Custom

Search actions...

Add role to access token

Klicke auf «Apply». Logge dich danach neu ein bei der Freelancer-Applikation, verwende den Benutzer mit der Admin-Rolle und kopiere das JWT aus der Konsole. Prüfe auf <https://jwt.io/>, ob die Rolle nun im JWT enthalten ist.

Encoded

PASTE A TOKEN HERE

```
eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCIsImtpZCI6IiE2VTZwTHdEWn15WWVjQTlsdUhMeiJ9.eyJ1c2VyX3JvbGVzIjpibImFkbWluI10sIm5pY2tuYW1lIjoiaHR0cHM6Ly9zLmdyYXZhdGFyLmNvbS9hdmF0YXIVNmI1NzZhnjhM2ZlMzhMGNiMmZmZjM2MDM3NWlwZWw_cz00ODAmcj1wZyZkPW0dHBzJTNBJTJGJTJGY2RuLmF1dGgwLmNvbSUyRmF2YXRhcnMlMkZnci5wbmciLCJ1cGRhdGV
```

Decoded

EDIT THE PAYLOAD AND SECRET

HEADER: ALGORITHM & TOKEN TYPE

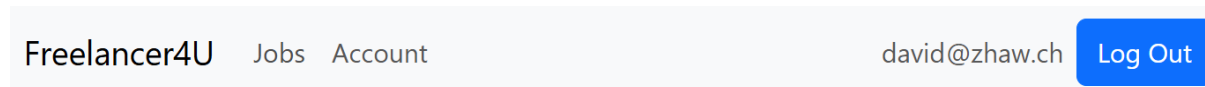
```
{  "alg": "RS256",  "typ": "JWT",  "kid": "Q6U6pLwDZyyYecA9luHLz"}
```

PAYLOAD: DATA

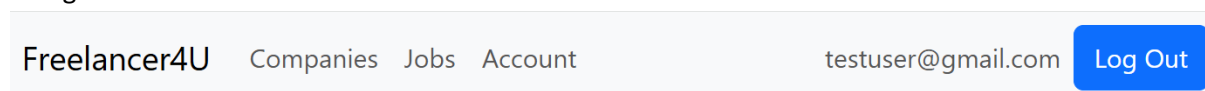
```
{  "user_roles": [    "admin"  ],  "nickname": "grud",  "name": "grud@zhaw.ch",
```

- d) Das Menu-Item «Company» soll nur noch angezeigt werden, wenn User die Rolle «admin» haben.

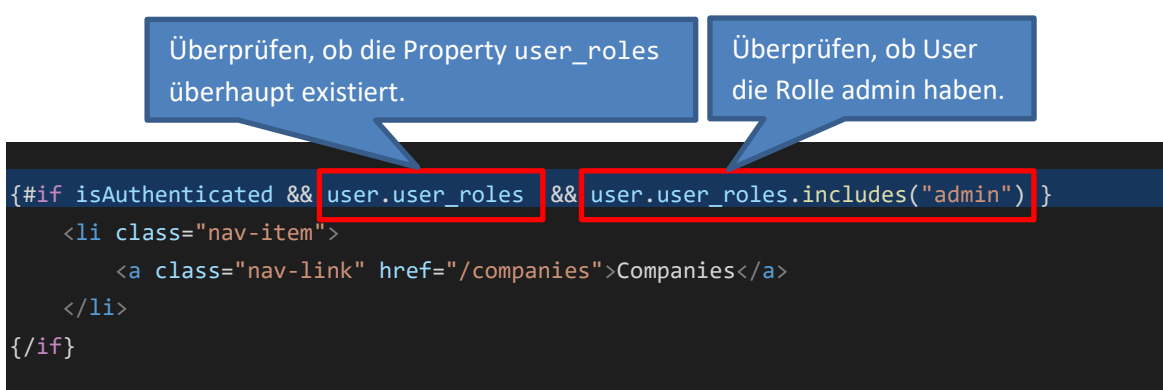
Navigation Bar ohne die Rolle «admin»:



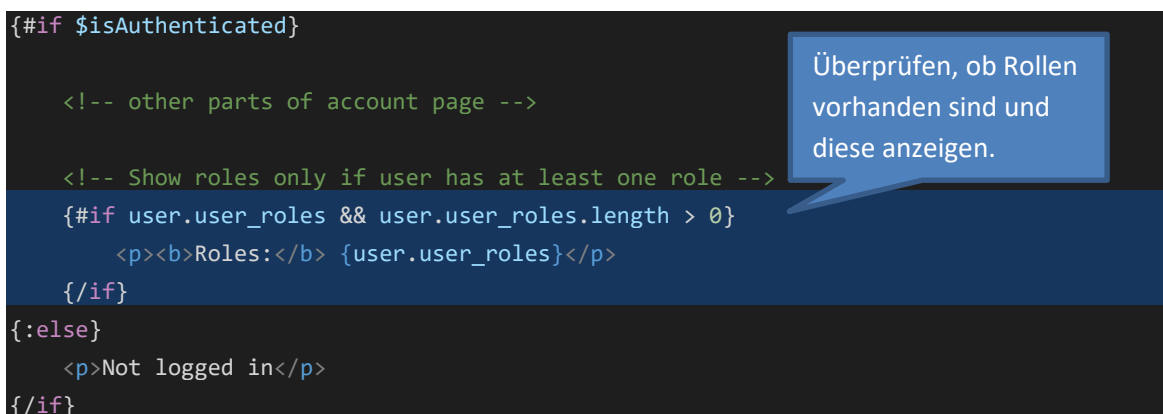
Navigation Bar mit der Rolle «admin»:



Dazu muss die folgende Bedingung in **routes/+layout.svelte** angepasst werden:



- e) Auch auf der Page «Account» sollen die Rollen angezeigt werden, sofern diese vorhanden sind:



- f) Auf der Page «Jobs» soll das Formular «Create Job» nur noch angezeigt werden, wenn der angemeldete User die Rolle «admin» hat. Wenn diese Rolle nicht vorhanden ist, wird nur die Liste aller Jobs angezeigt:

The screenshot shows the Freelancer4U interface. At the top, there are links for 'Freelancer4U', 'Jobs', and 'Account'. On the right, the user 'david.zhaw.ch' is logged in, with a 'Log Out' button. The main heading is 'All Jobs'. Below it is a table with the following data:

Description	Type	Earnings	State	CompanyId	FreelancerId
Design a database schema for an e-commerce platform	OTHER	556.4	ASSIGNED	31	ada@gmail.com
Build a RESTful API for a blog application	TEST	418.5	DONE	83	ada@gmail.com
Write unit tests for a shopping cart functionality	REVIEW	651.5	NEW	57	null
Design a database schema for an e-commerce platform	REVIEW	684.2	NEW	58	null
Design a database schema for an e-commerce platform	REVIEW	305.5	NEW	96	null
Write unit tests for a shopping cart functionality	IMPLEMENT	131.5	NEW	12	null
Build a RESTful API for a blog application	IMPLEMENT	349.7	NEW	63	null
Build a RESTful API for a blog application	OTHER	83.4	NEW	86	null
Design a database schema for an e-commerce platform	REVIEW	968	NEW	52	null

Callouts from the image:

- 'User ohne Admin-Rolle' points to the user profile area.
- 'Create Job wird nicht angezeigt' points to the area where the 'Create Job' button would be.

Passe den Code in *routes/jobs/+page.svelte* dementsprechend an. **Tipp:** Orientiere dich am bestehenden Code in *+layout.svelte*.

- g) Nun muss auch noch das Backend geschützt werden, so dass User ohne Admin-Rolle keine Jobs erstellen können. Weil Rollen an verschiedenen Orten geprüft werden müssen, erstellen wir einen Service. Füge dem Backend im Ordner «Service» die unten abgebildete Klasse *UserService.java* hinzu.

```
package ch.zhaw.freelancer4u.service;

import java.util.List;
import org.springframework.stereotype.Service;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.oauth2.jwt.Jwt;

@Service
public class UserService {

    public boolean userHasRole(String role) {
        Jwt jwt = (Jwt) SecurityContextHolder.getContext().getAuthentication().getPrincipal();
        List<String> userRoles = jwt.getClaimAsStringList("user_roles");
        if (userRoles.contains(role)) {
            return true;
        }
        return false;
    }
}
```

Erklärung: Im SecurityContextHolder speichert Spring Security die Details darüber, wer authentifiziert ist. In der Methode «hasRole» wird aus dem JWT das Feld «user_roles» ausgelesen und mit dem Input-Parameter «role» verglichen. Immer wenn die gewünschte Rolle enthalten ist, wird true zurückgegeben, sonst false. Dieser Service kann nun im **JobController** verwendet werden.

```
package ch.zhaw.freelancer4u.controller;

import java.util.List;
import org.springframework.beans.factory.annotation.Autowired;
import ...
import ch.zhaw.freelancer4u.service.UserService;

@RestController
@RequestMapping("/api")
public class JobController {

    @Autowired
    JobRepository jobRepository;

    @Autowired
    CompanyService companyService;

    @Autowired
    UserService userService;

    @PostMapping("/job")
    public ResponseEntity<Job> createJob(@RequestBody JobCreateDTO cDTO) {
        if (!userService.userHasRole("admin")) {
            return new ResponseEntity<>(HttpStatus.FORBIDDEN);
        }

        if (!companyService.companyExists(cDTO.getCompanyId())) {
            return new ResponseEntity<>(HttpStatus.BAD_REQUEST);
        }

        Job jDAO = new Job(cDTO.getDescription(), cDTO.getJobType(),
                           cDTO.getEarnings(), cDTO.getCompanyId());
        Job j = jobRepository.save(jDAO);
        return new ResponseEntity<>(j, HttpStatus.CREATED);
    }
}
```

Zusätzlicher Import

Service einbinden

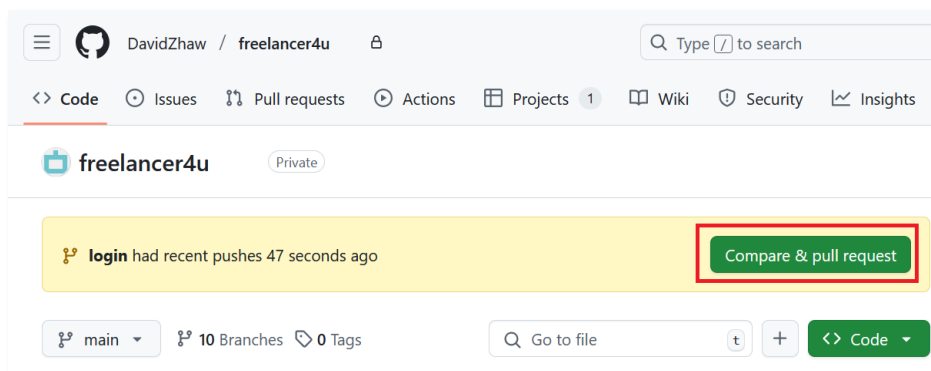
Nur die Rolle Admin darf einen POST-Request auf job ausführen

- h) Nun müssen auch noch alle GET und POST Endpoints im CompanyController abgesichert werden, so dass diese nur noch mit der Rolle «admin» aufgerufen werden können. Teste nochmal mit Postman, ob alles wie gewünscht funktioniert.

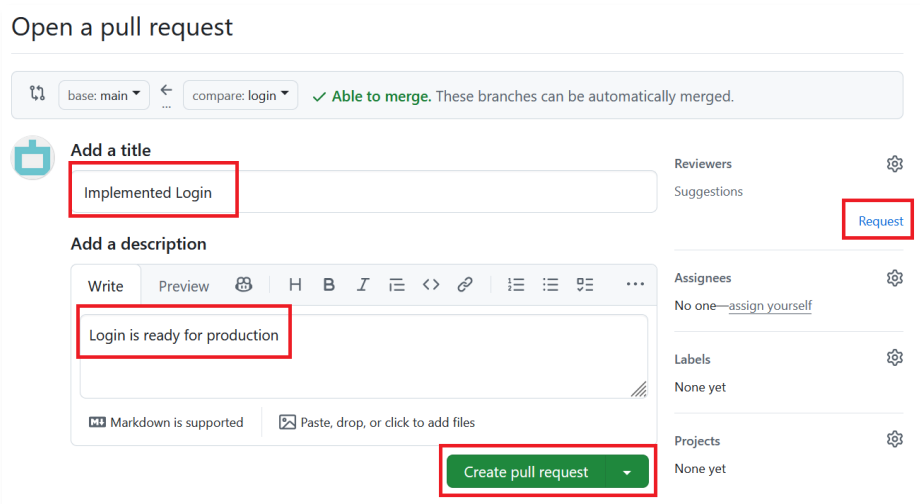
- i) Die Seite mit den Jobs funktioniert nun leider nicht mehr für normal User. Grund: sowohl für Admin wie auf normale User werden die Companies geladen. Benötigt werden diese aber nur im Admin-Formular zum Erfassern neuer Jobs. Passe die Load-Methode für Jobs in +page.server.js so an, dass Companies nur noch für Admin User geladen werden. **Tipp:** Die Rolle lässt sich gleich wie das Token aus den locals lesen.

```
const user_info = locals.user;
if (user_info.user_roles.includes('admin')) {
  // ...
}
```

- j) Committe anschliessend deine Änderungen auf GitHub (Commit-Message: «added roles to frontend and backend»).
- k) Nun wollen wir alle Änderungen aus dem Login-Branch in den Main-Branch mergen. Diesmal wollen wir das Mergen per **Pull-Request** üben. Dabei werden die Änderungen von einer anderen Person überprüft, bevor diese gemerged werden. Öffne dazu das Freelancer-Repo auf GitHub und füge eine(n) Mitstudierende(n) als Collaborator zu deinem Projekt hinzu.
- l) Erstelle einen Pull Request.



- m) Wir verzichten diesmal auf das Review. Wenn du willst, kannst du auch GitHub Copilot als Reviewer auswählen.



n) Bestätige den Pull-Request und merge die Änderungen.

The screenshot shows a GitHub Pull Request interface. At the top, the title is "Implemented Login #14". Below the title, a green button labeled "Open" is followed by the text "DavidZhaw wants to merge 3 commits into `main` from `login`". Below this, there are tabs for "Conversation" (0), "Commits" (3), "Checks" (0), and "Files changed" (18). The main content area shows a comment from "DavidZhaw" stating "Login is ready for production" with a smiley face emoji. Below the comment, a commit history is shown with three commits: "added login to frontend" (32f61b0), "added login to backend" (6cc6566), and "added roles to frontend and backend" (90c71ff). At the bottom, a green box contains a "Review requested" status, a "No conflicts with base branch" status, and a "Merge pull request" button. The "Merge pull request" button has a dropdown arrow and a link to "View command line instructions".

Implemented Login #14

Open DavidZhaw wants to merge 3 commits into `main` from `login`

Conversation 0 Commits 3 Checks 0 Files changed 18

DavidZhaw commented now

Login is ready for production

DavidZhaw added 3 commits 1 hour ago

- added login to frontend 32f61b0
- added login to backend 6cc6566
- added roles to frontend and backend 90c71ff

DavidZhaw requested a review from mmeisterhans now

Review requested

Review has been requested on this pull request. It is not required to merge. [Learn more about requesting a pull request review.](#)

1 pending review >

No conflicts with base branch

Merging can be performed automatically.

Merge pull request You can also merge this with the command line. [View command line instructions.](#)

Uff. Das war eine lange Übung 😊. Gib wiederum die URL deines Repos auf Moodle ab.